

Influencer Underwriting: Phase 0 Bootstrap

Purpose

This guide converts the current setup into the first reproducible project stage:

- Google Drive remains the source of truth for raw data.
- GitHub contains code, configuration, tests, and documentation.
- Colab provides temporary CPU/GPU compute.
- `/content` is used for fast temporary work.
- Derived Parquet and reports are copied back to Drive.

Do not commit the dataset or extracted images to GitHub.

1. Current situation

Your current Colab code does four things:

1. Mounts Google Drive.
2. Locates the dataset folder.
3. Creates `/content/data`.
4. Copies the light-tier files into the Colab runtime.

That is reasonable for an audit, but it should become a reusable bootstrap function rather than code embedded inside one large notebook.

The screenshot indicates that the dataset is currently visible in Google Drive under a shared dataset folder. The important thing is that Colab resolves the folder to a stable path. The first cell should print the resolved path and fail clearly if it cannot find the required files.

The JSON archive is approximately 3 GB. The image archives are much larger and should not be copied or extracted during Phase 0.

2. Recommended Drive layout

Create this structure inside a durable Drive folder, preferably in `MyDrive` :

Plain Text

```
InfluencerUnderwriting/  
├─ raw/
```

```
| | | | post_info.txt
| | | | json_files.zip
| | | | profiles_influencers.zip
| | | | profiles_brands.zip
| | | | img_fi01.zip ... img_fi16.zip
| | | | sample_images.zip
| | | |
| | | | artifacts/
| | | | | bronze/
| | | | | silver/
| | | | | embeddings/
| | | | | models/
| | | | | evaluations/
| | | | | reports/
| | | | |
| | | | checkpoints/
```

If moving shared files is not possible, keep the existing dataset location and create a separate `InfluencerUnderwriting/artifacts` folder in `MyDrive`. The code can use two paths:

Plain Text

```
RAW_DRIVE = existing dataset folder
ARTIFACT_DRIVE = MyDrive/InfluencerUnderwriting/artifacts
```

Never overwrite the raw archives from processing code.

3. Clone the repository inside Colab

Do not run the project directly from a Drive-mounted Git repository. Clone the repository into the fast local runtime:

Python

```
!rm -rf /content/influencer-underwriting
!git clone https://github.com/VAL-Jerono/influencer-underwriting.git \
    /content/influencer-underwriting

%cd /content/influencer-underwriting
!git status
```

Use Drive only for raw data and durable generated artifacts. Use GitHub for:

- Python source;
- notebooks;
- configuration;

- tests;
- README;
- model documentation;
- small example data only.

Do not commit:

- ZIP archives;
- raw JSON;
- raw images;
- extracted image folders;
- email or phone fields;
- model checkpoints larger than your repository policy permits.

4. Improved Colab bootstrap cell

Replace the original setup with a version that separates raw data, temporary data, code, and artifacts.

Python

```
from google.colab import drive
drive.mount('/content/drive' )

from pathlib import Path
import os
import shutil
import json
import hashlib

# Fast local code and temporary compute locations.
PROJECT_ROOT = Path('/content/influencer-underwriting')
LOCAL_ROOT = Path('/content/influencer_data')
LOCAL_RAW = LOCAL_ROOT / 'raw'
LOCAL_WORK = LOCAL_ROOT / 'work'

# Durable Drive locations.
ARTIFACT_ROOT = Path('/content/drive/MyDrive/InfluencerUnderwriting/artifacts')
CHECKPOINT_ROOT = Path('/content/drive/MyDrive/InfluencerUnderwriting/checkpoint')

for p in [LOCAL_RAW, LOCAL_WORK, ARTIFACT_ROOT, CHECKPOINT_ROOT]:
    p.mkdir(parents=True, exist_ok=True)
```

```

# Existing dataset location. Change only this value if the folder differs.
RAW_DRIVE = Path('/content/drive/MyDrive/Influencer brand dataset')

required = [
    'post_info.txt',
    'json_files.zip',
    'profiles_influencers.zip',
    'profiles_brands.zip',
]

missing = [name for name in required if not (RAW_DRIVE / name).exists()]

if missing:
    print('The expected MyDrive path was not found.')
    print('Missing:', missing)
    print('Search manually or create a MyDrive shortcut to the shared folder.')
    raise FileNotFoundError(RAW_DRIVE)

print('RAW_DRIVE =', RAW_DRIVE)
print('PROJECT_ROOT =', PROJECT_ROOT)
print('ARTIFACT_ROOT =', ARTIFACT_ROOT)

# Copy only light-tier inputs to the local runtime.
# Image archives are intentionally excluded.
for name in required:
    source = RAW_DRIVE / name
    destination = LOCAL_RAW / name
    complete_marker = destination.with_suffix(destination.suffix + '.complete')

    if complete_marker.exists() and destination.exists():
        print('already copied:', name)
        continue

    partial = destination.with_suffix(destination.suffix + '.part')
    if partial.exists():
        partial.unlink()

    print(f'copying {name} ...')
    shutil.copyfile(source, partial)
    partial.rename(destination)
    complete_marker.write_text('complete\n')
    print('copied:', name, destination.stat().st_size, 'bytes')

print('Local files:')
for p in sorted(LOCAL_RAW.iterdir()):
    print(p.name, p.stat().st_size)

```

Why this is better

- The code repository is local and fast.
- Temporary data is separated from code.
- Durable outputs go to Drive.
- Raw archives are not modified.
- A `.part` file prevents an interrupted copy from being mistaken for a complete file.
- A `.complete` marker prevents unnecessary repeated copies in the same Drive artifact area.
- Image archives are not copied.

Colab runtimes are temporary, so a new runtime may still need to copy the light-tier files again. The important point is that the processed Parquet outputs and checkpoints should be written to Drive and reused.

5. First repository structure to add

Create this structure in the GitHub repository:

Plain Text

```
influencer-underwriting/
├── README.md
├── requirements.txt
├── .gitignore
├── configs/
│   └── paths.example.yaml
├── src/
│   ├── underwriting/
│   │   ├── __init__.py
│   │   ├── io/
│   │   │   └── paths.py
│   │   ├── extraction/
│   │   │   └── json_stream.py
│   │   ├── schemas/
│   │   │   └── post_schema.py
│   │   ├── features/
│   │   └── monitoring/
├── scripts/
│   ├── audit_raw.py
│   └── extract_metadata.py
├── notebooks/
└── 01_audit_report.ipynb
```

```
├── tests/
│   └── test_smoke.py
└── docs/
    └── data_dictionary.md
```

The first notebook should call `scripts/audit_raw.py` or import functions from `src/underwriting` . It should not contain the complete extraction implementation.

6. First deliverable: raw audit manifest

Before extracting all JSON metadata, create a small audit manifest containing:

Plain Text

```
raw file name
file size
archive member count
post row count
post ID range
sponsorship counts
unique usernames
profile coverage
JSON coverage
image count distribution
JSON layout frequencies
comment-text coverage
category counts
```

Save two outputs:

Plain Text

```
Drive/InfluencerUnderwriting/artifacts/reports/raw_audit.json
Drive/InfluencerUnderwriting/artifacts/reports/raw_audit.md
```

The notebook should display the results, but the JSON and Markdown files are the durable outputs.

7. Do not process images in Phase 0

The first phase should not:

- copy image archives to `/content` ;

- extract the image archives;
- generate image embeddings;
- train a CNN;
- build a multimodal model.

First complete:

1. raw audit;
2. metadata extraction;
3. Parquet conversion;
4. feature table creation;
5. classical baselines;
6. leakage-safe evaluation.

Only then use a GPU for a stratified image sample. This prevents a large computational investment before knowing whether images improve the results.

8. First commands to run in Colab

After cloning and running the bootstrap cell:

Python

```
%cd /content/influencer-underwriting

!python -V
!pip install -q pyarrow duckdb polars scikit-learn pyyaml tqdm

!git status
!find . -maxdepth 3 -type f | sort | sed -n '1,120p'
```

Then run a lightweight audit against the local copies:

Python

```
import pandas as pd
from pathlib import Path

local_raw = Path('/content/influencer_data/raw')
post_info = local_raw / 'post_info.txt'

cols = ['post_id', 'username', 'sponsored', 'json_file', 'image_files']
```

```
posts = pd.read_csv(
    post_info,
    sep='\t',
    header=None,
    names=cols,
    dtype=str,
    quoting=3,
    keep_default_na=False,
    nrows=100_000,
)

print(posts.head())
print('sample rows:', len(posts))
print('sponsored counts:')
print(posts['sponsored'].value_counts(dropna=False))
print('unique usernames:', posts['username'].nunique())
```

This is only a smoke test. Do not confuse it with the complete audit. The full audit should process the file in chunks.

9. Immediate next milestone

The next implementation milestone should be:

Build a streaming metadata extractor that converts the four light-tier inputs into compact Parquet tables without touching the image archives.

Expected outputs:

Plain Text

```
artifacts/bronze/posts_metadata.parquet
artifacts/bronze/profiles_influencers.parquet
artifacts/bronze/profiles_brands.parquet
artifacts/bronze/post_image_index.parquet
artifacts/reports/raw_audit.json
artifacts/reports/raw_audit.md
```

Only after this milestone should you build `creator_features.parquet` and begin modeling.

10. Final operating rule

Your setup should eventually behave like this:

Plain Text

GitHub

= source code and reproducible logic

Drive/raw

= immutable source data

Drive/artifacts

= durable Parquet, embeddings, models, reports, checkpoints

Colab /content

= temporary high-speed compute

Notebook

= orchestration, explanation, and visualization

Scripts and src/

= reusable pipeline implementation

API/dashboard

= small serving layer over compact artifacts

This lets you work with the multi-gigabyte dataset without overwhelming your laptop and without repeatedly rebuilding the project from scratch after a Colab disconnect.